Практикум: ETL с Apache Spark

Spark → Star Schema → ClickHouse

Цель: спроектировать и реализовать ETL-конвейер на базе Apache Spark для трансформации денормализованных данных о продажах зоотоваров в модель «звезда» и аналитические витрины в ClickHouse.

2026 г.

1 Постановка задачи

1.1 Исходные данные

Из лабораторной работы №1 вам доступны 10 CSV-файлов (`mock_data_*.csv`) с данными о продажах зоотоваров. Каждый файл содержит 1000 строк, итого 10 000 записей. Формат данных — полностью денормализованная таблица со следующими группами полей:

Группа полей	Примеры
Идентификатор	`id` (1–1000 в каждом файле)
Данные клиента	`customer_first_name`, `customer_last_name`, `customer_age`, `customer_email`, `customer_country`, `customer_postal_code`, `customer_pet_type`, `customer_pet_name`, `customer_pet_breed`
Данные продавца	`seller_first_name`, `seller_last_name`, `seller_email`, `seller_country`, `seller_postal_code`
Данные товара	`product_name`, `product_category`, `product_price`, `product_quantity`, `pet_category`, `product_weight`, `product_color`, `product_size`, `product_brand`, `product_material`, `product_description`, `product_rating`, `product_reviews`, `product_release_date`, `product_expiry_date`
Данные продажи	`sale_date`, `sale_customer_id`, `sale_seller_id`, `sale_product_id`, `sale_quantity`, `sale_total_price`
Данные магазина	`store_name`, `store_location`, `store_city`, `store_state`, `store_country`, `store_phone`, `store_email`
Данные поставщика	`supplier_name`, `supplier_contact`, `supplier_email`, `supplier_phone`, `supplier_address`, `supplier_city`, `supplier_country`

Внимание: Все 10 файлов содержат одинаковый диапазон `id` от 1 до 1000. При загрузке в единую таблицу потребуется обработка пересекающихся идентификаторов — см. этап 2.

1.2 Что нужно построить

Двухэтапный ETL-конвейер на базе Apache Spark:

1. Импорт CSV в PostgreSQL (staging-таблица).
2. Spark-приложение: чтение из staging → трансформация в модель «звезда» → запись в PostgreSQL.
3. Spark-приложение: чтение модели «звезда» → вычисление 6 аналитических витрин → запись в ClickHouse.

Архитектура (общий вид):

```
┌──────────────┐     ┌──────────────────┐     ┌──────────────┐     ┌──────────────┐
│  CSV-файлы   │──►│  PostgreSQL      │──►│   Spark      │──►│  PostgreSQL  │
│ (10 файлов)  │     │  (staging, 10k)  │     │  (ETL Job)   │     │ (Star Schema)│
└──────────────┘     └──────────────────┘     └──────────────┘     └──────────────┘
                                                                          │
```

```
        ┌──────────┴──┐
        ▼             ▼
┌──────────────┐  ┌──────────────┐
│    Spark     │  │ClickHouse│
│(Reports)     │→│(6 отчётов)│
└──────────────┘  └──────────────┘
```

Все компоненты запускаются через Docker Compose.

1.3 Шесть обязательных витрин (отчётов)

№	Витрина	Содержание
1	Продукты	Топ-10 продаваемых, выручка по категориям, средний рейтинг
2	Клиенты	Топ-10 по сумме покупок, распределение по странам, средний чек
3	Время	Месячные/годовые тренды, сравнение периодов, средний заказ по месяцам
4	Магазины	Топ-5 по выручке, продажи по городам/странам, средний чек
5	Поставщики	Топ-5 по выручке, средняя цена товаров, продажи по странам
6	Качество	Max/min рейтинг, корреляция рейтинга и продаж, число отзывов

Вопрос для самопроверки: Почему витрины данных целесообразно вычислять именно из модели «звезда», а не из staging-таблицы?

1.4 Требования к результату

1. `docker-compose.yml` с PostgreSQL + Spark + ClickHouse (обязательно).
2. Инструкция по запуску Spark-джоб.
3. Spark-код: staging → star schema в PostgreSQL.
4. Spark-код: star schema → 6 отчётов в ClickHouse.
5. Опционально: Spark-код для Cassandra, Neo4j, MongoDB, Valkey.

2 Краткая теория

2.1 Apache Spark

Распределённый движок обработки данных. В отличие от Flink (потоковый), Spark исторически ориентирован на **пакетную** (batch) обработку, хотя с версии 2.x поддерживает Structured Streaming.

Основные абстракции:

- **`SparkSession`** — точка входа. Создаётся один раз на приложение. Через неё происходит чтение данных, создание DataFrame и доступ к SparkContext.
- **`DataFrame`** — распределённая коллекция строк с именованными колонками. Аналог таблицы в РСУБД. Неизменяемая (immutable).
- **Lazy evaluation** — преобразования (`select`, `filter`, `groupBy`, `join`) не выполняются немедленно. Они формируют план выполнения, который запускается только при действии (`show`, `count`, `write`, `collect`).
- **Partition** — DataFrame разбит на разделы, обрабатываемые параллельно на разных узлах.

Чтение и запись через JDBC:

```
df = spark.read \
    .format("jdbc") \
    .option("url", "jdbc:postgresql://host:5432/db") \
    .option("dbtable", "schema.table") \
    .option("user", "user") \
    .option("password", "pass") \
    .option("driver", "org.postgresql.Driver") \
    .load()

df.write \
    .format("jdbc") \
    .option("url", "...") \
    .option("dbtable", "target_table") \
    .mode("append") \
    .save()
```

Подсказка: Для работы JDBC Spark нужен соответствующий JAR-файл драйвера. Его следует скачать и указать через `--jars` при вызове `spark-submit` или поместить в каталог `jars/` Spark. Для PostgreSQL: `postgresql-42.7.x.jar`, для ClickHouse: `clickhouse-jdbc-0.6.x.jar` (или официальный Spark-коннектор).

2.2 Схема «звезда» (Star Schema)

Паттерн проектирования хранилищ данных:

- **Таблицы измерений** (dimension) — описательные атрибуты: кто, что, где. Пример: `dim_customer` с полями имени, возраста, страны.
- **Таблица фактов** (fact) — измеримые события (продажи) с числовыми мерами (сумма, количество) и внешними ключами на измерения.

```
        dim_customer
             |
```

```
        dim_seller
              |
dim_store —— fact_sales —— dim_product
              |
       dim_supplier
```

Подсказка: Для клиента, продавца и товара можно использовать `id` из CSV. Для магазина и поставщика уникального ID нет — придётся генерировать суррогатный ключ (например, `monotonically_increasing_id()` или хеш от полей).

2.3 ClickHouse

Колонно-ориентированная СУБД для аналитических запросов (OLAP). Ключевые отличия от PostgreSQL:

- Данные хранятся **по колонкам**, что ускоряет агрегации по отдельным полям.
- Основной движок таблиц — **MergeTree**: данные сортируются по указанному ключу и объединяются в фоне.
- Запись — преимущественно **пакетная** (batch INSERT). Частые построчные вставки неэффективны.
- Нет полноценной поддержки транзакций и внешних ключей.

```cpp
CREATE TABLE report_products (
    product_name String,
    category     String,
    total_sold   UInt64,
    revenue      Decimal(12, 2),
    avg_rating   Float32
) ENGINE = MergeTree()
ORDER BY (category, product_name);
```

Подсказка: Каждая таблица в ClickHouse **обязана** иметь `ENGINE = MergeTree()` и `ORDER BY`. Поле из `ORDER BY` становится первичным ключом (primary key) по умолчанию. Выбирайте ключи, по которым будут идти фильтры и группировки в запросах.

Вопрос для самопроверки: Почему ClickHouse лучше подходит для аналитических отчётов, чем PostgreSQL? В чём его ограничения по сравнению с PostgreSQL?

3 Этап 1. Опишите инфраструктуру в Docker Compose

Задание: Создайте `docker-compose.yml` с сервисами: PostgreSQL, Spark (master + worker или standalone с Jupyter), ClickHouse. Опционально: Cassandra, Neo4j, MongoDB, Valkey.

3.1 Сервис PostgreSQL

1. Образ: `postgres:16`.
2. Задайте `POSTGRES_DB`, `POSTGRES_USER`, `POSTGRES_PASSWORD`.
3. Пробросьте порт 5432 на хост.
4. Примонтируйте SQL-скрипт инициализации в `/docker-entrypoint-initdb.d/`.
5. Добавьте healthcheck (`pg_isready`).

Подсказка: Скрипт инициализации должен создать:

- Staging-таблицу `mock_data` (все колонки из CSV).
- Таблицы модели «звезда» (`dim_customer`, `dim_seller`, `dim_product`, `dim_store`, `dim_supplier`, `fact_sales`).

3.2 Сервис Spark

Есть два варианта организации:

3.2.1 Вариант A: Spark Master + Worker

1. Два сервиса: `spark-master` и `spark-worker`.
2. Образ: `bitnami/spark:3.5` или `apache/spark:3.5.0`.
3. Master: порт 8080 (UI), порт 7077 (кластер).
4. Worker: зависит от master.
5. Оба: примонтируйте JAR-файлы JDBC-драйверов и ваш код.

3.2.2 Вариант B: Spark + Jupyter (рекомендуется для разработки)

1. Один сервис на базе образа `jupyter/pyspark-notebook` или кастомного образа с Spark + Jupyter.
2. Пробросьте порт 8888 (Jupyter).
3. Примонтируйте директорию проекта.

Подсказка: В любом варианте JDBC JAR-файлы (PostgreSQL, ClickHouse) должны быть доступны Spark. Укажите их через `--jars` в `spark-submit` или положите в `$SPARK_HOME/jars/`. При монтировании из хоста используйте volume.

Внимание: Spark-контейнер должен находиться в одной Docker-сети с PostgreSQL и ClickHouse. Используйте имена сервисов как DNS-имена хостов в JDBC URL (например, `jdbc:postgresql://postgres:5432/mydb`).

3.3 Сервис ClickHouse

1. Образ: `clickhouse/clickhouse-server:24.8` (или аналогичный).
2. Пробросьте порт 8123 (HTTP) и/или 9000 (native).

3. Примонтируйте SQL-скрипт создания таблиц отчётов (аналогично PostgreSQL, можно через `/docker-entrypoint-initdb.d/`).
4. Добавьте healthcheck (`clickhouse-client --query 'SELECT 1'`).

Подсказка: Скрипт инициализации ClickHouse должен создать 6 таблиц для витрин. Каждая таблица — `ENGINE = MergeTree() ORDER BY (...)`. Колонки витрин определяются самостоятельно на основе требований к отчётам (см. раздел «6 обязательных витрин»).

3.4 Проверка сети

После запуска всех сервисов:

```
docker-compose up -d
docker exec -it <spark-container> ping -c 2 postgres
docker exec -it <spark-container> ping -c 2 clickhouse
```

Контрольная точка: Все контейнеры должны быть в статусе `healthy` / `running`. Spark должен пинговать PostgreSQL и ClickHouse по именам сервисов.

4 Этап 2. Импорт CSV в PostgreSQL (staging)

Задание: Загрузите все 10 CSV-файлов в единую staging-таблицу `mock_data` в PostgreSQL. Итого — 10 000 строк.

4.1 Создание staging-таблицы

Определите SQL-DDL для таблицы `mock_data` со всеми колонками из CSV. Типы данных:

1. `INTEGER` — для `id`, `customer_age`, `product_quantity`, `sale_quantity`, `product_reviews`, `sale_customer_id`, `sale_seller_id`, `sale_product_id`.
2. `DECIMAL(10,2)` — для `product_price`, `sale_total_price`, `product_weight`.
3. `VARCHAR(n)` — для строковых полей.
4. `DATE` — для `sale_date`, `product_release_date`, `product_expiry_date`.
5. `TEXT` — для `product_description`.

Подсказка: Все CSV-колонки можно объявить как `TEXT` в staging — строгая типизация будет в таблицах модели «звезда». Это упрощает импорт и позволяет обрабатывать ошибки преобразования типов на этапе Spark-трансформации.

4.2 Проблема пересекающихся ID

Внимание: Все 10 CSV-файлов содержат одинаковый диапазон `id` от 1 до 1000. При прямой загрузке в единую таблицу возникнут дубликаты по `id` или конфликты первичного ключа.

Решение: добавьте смещение (offset) при загрузке каждого файла:

1. Файл 0: offset = 0 → id 1–1000
2. Файл 1: offset = 1000 → id 1001–2000
3. …
4. Файл 9: offset = 9000 → id 9001–10000

Смещать нужно не только `id`, но и `sale_customer_id`, `sale_seller_id`, `sale_product_id`.

Подсказка: Способы загрузки: (1) `COPY` через psql для каждого файла с последующим `UPDATE` для offset; (2) скрипт на Python, который читает CSV, добавляет offset и вставляет через `INSERT`; (3) использовать `\copy` или `COPY` в DBeaver поочерёдно для каждого файла, предварительно обновив offset.

Вопрос для самопроверки: Что произойдёт, если загрузить все файлы без offset? Сколько уникальных записей окажется в staging-таблице?

Контрольная точка: После загрузки: `SELECT COUNT(*) FROM mock_data;` должно вернуть 10 000.

5 Этап 3. Spark: staging → модель «звезда» в PostgreSQL

Задание: Напишите Spark-приложение, которое читает данные из staging-таблицы `mock_data` в PostgreSQL и записывает их в таблицы модели «звезда» в том же PostgreSQL.

5.1 Структура приложения

Рекомендуемый порядок разработки:

5.1.1 1. Создание SparkSession

Настройте `SparkSession` с appName и, при необходимости, укажите master.

Подсказка:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("StagingToStarSchema") \
    .config("spark.jars", "/path/to/postgresql-42.7.3.jar") \
    .getOrCreate()
```

5.1.2 2. Чтение staging-таблицы

Используйте `spark.read.format("jdbc")` с параметрами подключения к PostgreSQL.

Подсказка: Для выноса параметров подключений используйте переменные окружения или файл конфигурации. Определите вспомогательную функцию для построения JDBC-опций.

5.1.3 3. Формирование таблиц измерений

Для каждого измерения:

1. Выберите нужные колонки (`select`).
2. Уберите дубликаты (`dropDuplicates` по ключевым полям).
3. Сгенерируйте суррогатный ключ, если нет естественного.
4. Запишите в таблицу измерения в PostgreSQL через `df.write.format("jdbc")`.

Таблицы измерений и их ключевые поля:

Таблица	Первичный ключ	Источник ключа
`dim_customer`	`customer_id`	`id` из CSV (с offset)
`dim_seller`	`seller_id`	`id` из CSV (с offset)
`dim_product`	`product_id`	`id` из CSV (с offset)
`dim_store`	`store_id`	Суррогатный (хеш или `monotonically_increasing_id`)
`dim_supplier`	`supplier_id`	Суррогатный (хеш или `monotonically_increasing_id`)

Внимание: Для `dim_store` и `dim_supplier` в CSV нет уникального ID. Один и тот же магазин/поставщик может встречаться в разных строках. Нужно выделить уникальные записи по комбинации полей и сгенерировать ID.

Подсказка: Варианты генерации суррогатного ключа: (1) `pyspark.sql.functions.monotonically_increasing_id()` — простой, но недетерминированный; (2) хеш от конкатенации ключевых полей — детерминированный, воспроизводимый при перезапуске.

5.1.4 4. Формирование таблицы фактов

1. Выберите меры: `sale_quantity`, `sale_total_price`, `sale_date`.
2. Добавьте внешние ключи: `customer_id`, `seller_id`, `product_id`, `store_id`, `supplier_id`.
3. Для `store_id` и `supplier_id` потребуется `JOIN` с соответствующими измерениями (чтобы сопоставить атрибуты магазина/поставщика с их суррогатными ключами).

Подсказка: Порядок записи: сначала все таблицы измерений, затем `fact_sales`. Это гарантирует, что внешние ключи ссылаются на существующие записи.

Вопрос для самопроверки: Почему важно записывать таблицы измерений до таблицы фактов? Что произойдёт при нарушении этого порядка, если включены внешние ключи?

Контрольная точка: Проверьте количество записей после записи:

```
SELECT 'fact_sales' AS t, COUNT(*) FROM fact_sales UNION ALL
SELECT 'dim_customer', COUNT(*) FROM dim_customer UNION ALL
SELECT 'dim_product', COUNT(*) FROM dim_product UNION ALL
SELECT 'dim_seller', COUNT(*) FROM dim_seller UNION ALL
SELECT 'dim_store', COUNT(*) FROM dim_store UNION ALL
SELECT 'dim_supplier', COUNT(*) FROM dim_supplier;
```

Ожидается: `fact_sales` = 10 000, таблицы измерений ≤ 10 000 каждая (с учётом дубликатов).

6 Этап 4. Spark: star schema → отчёты в ClickHouse

Задание: Напишите Spark-приложение, которое читает таблицы модели «звезда» из PostgreSQL, вычисляет 6 аналитических витрин и записывает их в ClickHouse.

6.1 Общая схема приложения

```
┌──────────────────────┐
│  Spark reads from    │
│  PostgreSQL (star    │──► 6 DataFrames ──► 6 writes to ClickHouse
│  schema tables)      │
└──────────────────────┘
```

1. Создайте `SparkSession` с JDBC JAR-файлами для PostgreSQL **и** ClickHouse.
2. Прочитайте все таблицы измерений и фактов из PostgreSQL.
3. Для каждой витрины постройте DataFrame с агрегациями.
4. Запишите каждый DataFrame в соответствующую таблицу ClickHouse через JDBC.

Подсказка: ClickHouse JDBC URL: `jdbc:clickhouse://clickhouse:8123/default`. Driver class: `com.clickhouse.jdbc.ClickHouseDriver`. Используйте `df.write.format("jdbc")` аналогично записи в PostgreSQL.

Внимание: Перед записью убедитесь, что 6 таблиц витрин **уже созданы** в ClickHouse (DDL через скрипт инициализации или вручную). Spark JDBC `write` не создаёт таблицы автоматически с нужным движком MergeTree.

6.2 Витрина 1. Продукты (`report_products`)

1. **Топ-10 продаваемых**: `GROUP BY product_name`, `SUM(sale_quantity)`, `ORDER BY total_sold DESC`, `LIMIT 10`.
2. **Выручка по категориям**: `GROUP BY product_category`, `SUM(sale_total_price)`.
3. **Средний рейтинг**: `AVG(product_rating)`, `SUM(product_reviews)`.

Определите колонки таблицы самостоятельно. Рекомендуется объединить все три аспекта в одну таблицу или создать три отдельные таблицы.

Подсказка: Используйте `df.join(fact, product)` для связи фактов с товарами. Агрегации: `pyspark.sql.functions.sum`, `avg`, `count`, `round`. Для топ-N: `.orderBy(col("total_sold").desc())` и `.limit(10)`.

6.3 Витрина 2. Клиенты (`report_customers`)

1. **Топ-10 по сумме покупок**: `GROUP BY customer_id, name`, `SUM(sale_total_price)`.
2. **Распределение по странам**: `GROUP BY customer_country`, `COUNT(DISTINCT customer_id)`.
3. **Средний чек**: `SUM(sale_total_price) / COUNT(sale_id)` для каждого клиента.

Подсказка: Для «среднего чека» используйте `avg("sale_total_price")` по клиенту — это среднее значение одного заказа.

6.4 Витрина 3. Время (`report_time`)

1. **Месячные тренды**: извлеките год и месяц из `sale_date` (`pyspark.sql.functions.year`, `month`), `GROUP BY year, month`, `SUM(sale_total_price)`.
2. **Сравнение периодов**: выручка по годам.
3. **Средний заказ по месяцам**: `AVG(sale_total_price)` группировка по году и месяцу.

Подсказка: Для извлечения компонентов даты: `F.col("sale_date").cast("date")`, затем `F.year(...)`, `F.month(...)`. Убедитесь, что `sale_date` корректно парсится из staging.

6.5 Витрина 4. Магазины (`report_stores`)

1. **Топ-5 по выручке**: `GROUP BY store_name`, `SUM(sale_total_price)`, `ORDER BY revenue DESC`, `LIMIT 5`.
2. **Продажи по городам/странам**: `GROUP BY store_city, store_country, COUNT(*), SUM(sale_total_price)`.
3. **Средний чек**: `AVG(sale_total_price)` по магазину.

Подсказка: Понадобится JOIN `fact_sales` с `dim_store` по `store_id`.

6.6 Витрина 5. Поставщики (`report_suppliers`)

1. **Топ-5 по выручке**: JOIN `fact_sales` с `dim_supplier` через `dim_product` (связь продукта и поставщика).
2. **Средняя цена товаров**: `AVG(product_price)` по поставщику.
3. **Продажи по странам поставщиков**: `GROUP BY supplier_country, SUM(sale_total_price)`.

Подсказка: Поскольку в исходных данных поставщик связан с продуктом в каждой строке CSV, связь `product ↔ supplier` уже заложена. Определите, как хранить `supplier_id` в `dim_product` или `fact_sales` на этапе 3.

6.7 Витрина 6. Качество (`report_quality`)

1. **Max/min рейтинг**: `ORDER BY product_rating DESC/ASC`, `LIMIT 1` (или `MAX/MIN` агрегация).
2. **Корреляция рейтинга и продаж**: JOIN `fact_sales` с `dim_product`, вычислите `SUM(sale_quantity)` и сравните с `product_rating`.
3. **Число отзывов**: `GROUP BY product_name`, `SUM(product_reviews)`, `ORDER BY total_reviews DESC`.

Подсказка: Для корреляции можно использовать `pyspark.sql.functions.corr("product_rating", "total_sold")` или вывести DataFrame с двумя колонками и проанализировать визуально.

Контрольная точка: После записи всех витрин проверьте в ClickHouse:

```
SELECT 'report_products', COUNT(*) FROM report_products UNION ALL
SELECT 'report_customers', COUNT(*) FROM report_customers UNION ALL
SELECT 'report_time', COUNT(*) FROM report_time UNION ALL
SELECT 'report_stores', COUNT(*) FROM report_stores UNION ALL
SELECT 'report_suppliers', COUNT(*) FROM report_suppliers UNION ALL
SELECT 'report_quality', COUNT(*) FROM report_quality;
```

Все таблицы должны быть непустыми.

7 Этап 5. Проверка отчётов в ClickHouse

Задание: Выполните проверочные запросы к каждой витрине и убедитесь, что данные корректны.

7.1 Подключение к ClickHouse

```
docker exec -it $(docker ps -q -f name=clickhouse) \
  clickhouse-client
```

Или через HTTP-интерфейс:

```
curl 'http://localhost:8123/?query=SELECT+1'
```

7.2 Проверочные запросы

Для каждой витрины выполните:

7.2.1 Витрина «Продукты»

```
-- Топ-10 должны быть с > 0 продажами
SELECT * FROM report_products ORDER BY total_sold DESC LIMIT 10;

-- Сумма выручки по категориям должна быть осмысленной
SELECT category, sum(revenue) FROM report_products GROUP BY category ORDER BY sum(revenue)
DESC;
```

7.2.2 Витрина «Клиенты»

```
-- Топ-10 клиентов — проверить, что суммы > 0
SELECT * FROM report_customers ORDER BY total_spent DESC LIMIT 10;

-- Количество стран > 1
SELECT count(DISTINCT country) FROM report_customers;
```

7.2.3 Витрина «Время»

```
-- Проверить, что есть данные за все месяцы
SELECT year, month, count FROM report_time ORDER BY year, month;
```

7.2.4 Витрина «Магазины»

```
-- Топ-5 магазинов
SELECT * FROM report_stores ORDER BY revenue DESC LIMIT 5;
```

7.2.5 Витрина «Поставщики»

```
-- Топ-5 поставщиков
SELECT * FROM report_suppliers ORDER BY revenue DESC LIMIT 5;
```

7.2.6 Витрина «Качество»

```
-- Продукты с max/min рейтингом
SELECT * FROM report_quality ORDER BY rating DESC LIMIT 1;
SELECT * FROM report_quality ORDER BY rating ASC LIMIT 1;
```

Контрольная точка: Все запросы должны возвращать непустые, осмысленные результаты. Если таблица пуста — проверьте: (1) existence of MergeTree table in ClickHouse before Spark write; (2) Spark logs for write errors; (3) JDBC connectivity between Spark and ClickHouse.

8 Этап 6 (опционально). Запись отчётов в другие NoSQL БД

Задание: Реализуйте запись тех же 6 витрин в Cassandra, Neo4j, MongoDB или Valkey. Каждый отчёт — отдельная таблица/коллекция/ключ.

8.1 Cassandra

1. Язык запросов: **CQL** (Cassandra Query Language).
2. Понятия: **keyspace** (аналог БД), **table**, **partition key**.
3. Подход: создайте keyspace и 6 таблиц через CQL, запишите DataFrame через `spark.write.format("org.apache.spark.sql.cassandra")`.

Подсказка: Требуется `spark-cassandra-connector`. В `docker-compose.yml` добавьте сервис Cassandra (порт 9042). Таблицы Cassandra требуют `PRIMARY KEY` — определите partition key исходя из паттернов доступа к отчёту.

8.2 Neo4j

1. Язык запросов: **Cypher**.
2. Понятия: **nodes** (узлы), **relationships** (связи), **properties** (атрибуты).
3. Подход: каждый отчёт можно представить как набор узлов с атрибутами витрины. Например, для «Продуктов» — узлы `Product` со свойствами `total_sold`, `revenue`, `avg_rating`.

Подсказка: Используйте коннектор `neo4j-spark-connector`. Альтернативно: записывайте данные через Cypher-запросы (`MERGE`), отправляемые из Spark через Neo4j Java driver. Порт: 7687 (Bolt).

8.3 MongoDB

1. Формат: **документы** (BSON/JSON) в **коллекциях**.
2. Подход: каждый отчёт — отдельная коллекция. DataFrame записывается как набор документов.

Подсказка: Используйте `spark.write.format("mongodb")` из `mongo-spark-connector`. Порт: 27017. Каждая строка DataFrame становится JSON-документом. Колонки DataFrame → поля документа.

8.4 Valkey (Redis-совместимый)

1. Модель: **ключ-значение** (key-value). Поддерживает хеши, списки, множества.
2. Подход: каждую строку отчёта можно хранить как Redis Hash: ключ `report:<название>:<id>`, поля — колонки витрины.

Подсказка: Для записи из Spark можно использовать `spark-redis` коннектор или написать собственный метод записи через `redis-py`. Порт Valkey: 6379. Пример ключа: `report:products:1` → hash с полями `product_name`, `category`, `total_sold`, `revenue`, `avg_rating`.

9 Типичные проблемы и решения

Симптом	Причина / решение
`ClassNotFoundException: org.postgresql.Driver`	JDBC JAR не найден Spark. Добавьте `--jars /path/to/postgresql-42.7.x.jar` в `spark-submit` или положите в `$SPARK_HOME/jars/`.
`ClassNotFoundException: com.clickhouse.jdbc.ClickHouseDriver`	Аналогично: добавьте ClickHouse JDBC JAR.
`Cannot connect to postgresql:5432`	Контейнеры в разных сетях. Убедитесь, что все сервисы в одном `networks:` в `docker-compose.yml`. Используйте DNS-имя сервиса, не `localhost`.
ClickHouse: `Cannot create table without engine`	Укажите `ENGINE = MergeTree() ORDER BY (...)` в CREATE TABLE. MergeTree — обязательный движок.
ClickHouse: `Too many parts`	Слишком частые построчные INSERT. Используйте пакетную запись (batch INSERT). Spark JDBC пишет батчами — проверьте `batchsize` option.
`relation "dim_customer" does not exist`	Таблицы star schema не созданы. Убедитесь, что SQL-скрипт инициализации PostgreSQL отработал. Проверьте `/docker-entrypoint-initdb.d/`.
В staging только 1000 строк вместо 10000	CSV-файлы имеют одинаковые `id`. Добавьте offset при загрузке каждого файла (file_index × 1000).
В fact_sales внешние ключи ссылаются на несуществующие записи	Смещение (offset) не добавлено к `sale_customer_id`, `sale_seller_id`, `sale_product_id`. Все FK-поля нужно сдвигать так же, как `id`.
`spark-submit: command not found`	Запуск `spark-submit` не внутри Spark-контейнера. Выполните через `docker exec`.
`Address already in use: 5432`	Порт PostgreSQL занят на хосте. Остановите локальный PostgreSQL или измените порт в `ports:` docker-compose.
Пустые витрины в ClickHouse	Таблицы ClickHouse не созданы до записи Spark, или ошибка записи проигнорирована. Проверьте логи Spark.

10 Контрольные вопросы

10.1 Apache Spark

1. Что такое `SparkSession`? Чем отличается от `SparkContext`?
2. Что такое DataFrame? Чем отличается от RDD?
3. Объясните концепцию **lazy evaluation** в Spark. Какие операции — transformations, какие — actions? Приведите примеры.
4. Что такое partition в Spark? Как количество разделов влияет на производительность?
5. Что делает `monotonically_increasing_id()`? Гарантирует ли она уникальность? Детерминирована ли она?
6. Как работает `dropDuplicates()`? В чём разница с `distinct()`?

10.2 JDBC и Spark

1. Как Spark читает данные из PostgreSQL через JDBC? Что происходит под капотом?
2. Какие параметры JDBC нужны для записи DataFrame в БД?
3. Что произойдёт при `mode("append")`, если таблица уже содержит данные? А при `mode("overwrite")`?
4. Зачем нужен JDBC JAR-файл? Что произойдёт, если его забыть?
5. Как настроить размер батча при записи через JDBC (параметр `batchsize`)? Почему это важно для ClickHouse?

10.3 ClickHouse

1. Чем ClickHouse отличается от PostgreSQL? Для каких задач предназначен каждый?
2. Что такое MergeTree-движок? Зачем нужен `ORDER BY`?
3. Почему ClickHouse не подходит для частых построчных INSERT?
4. Что будет, если создать таблицу без указания ENGINE?

10.4 Модель «звезда» и ETL

1. Чем таблица фактов отличается от таблицы измерений? Приведите примеры полей каждого типа.
2. Почему `fact_sales` нужно создавать после всех таблиц измерений?
3. Что такое суррогатный ключ? Когда он нужен и как его сгенерировать в Spark?
4. Почему витрины данных лучше вычислять из модели «звезда», а не из staging?

10.5 Отчёты и агрегации

1. Какие агрегатные функции Spark вы использовали для витрин? Назовите минимум пять.
2. Как получить топ-N записей в Spark DataFrame?
3. Что делает `groupBy().agg()`? Чем отличается от `groupBy().sum()`?
4. Как извлечь год и месяц из даты в Spark?
5. Как проверить корректность вычисленной витрины? Приведите конкретные критерии.

10.6 NoSQL (опционально)

1. В чём принципиальная разница между ClickHouse и Cassandra?
2. Какие данные естественно моделировать в Neo4j (графовая модель)?
3. Почему MongoDB удобна для хранения JSON-витрин?
4. Что такое Valkey? Чем отличается от «классической» РСУБД?

11 Справочник команд

11.1 Docker Compose

Команда	Описание
`docker-compose up -d --build`	Запустить все сервисы в фоне с пересборкой
`docker-compose up -d --build postgres spark-master spark-worker clickhouse`	Запустить только основные сервисы
`docker-compose down -v`	Остановить + удалить тома
`docker ps`	Запущенные контейнеры
`docker ps --format "table {{.Names}}\t{{.Status}}"`	Краткий статус контейнеров
`docker logs <container>`	Логи контейнера
`docker exec -it <id> bash`	Войти в контейнер

11.2 Spark (spark-submit)

Команда	Описание
`spark-submit --jars /path/pg.jar,/path/ch.jar --master local[*] app.py`	Запуск Spark-приложения локально с JDBC JAR
`spark-submit --master spark://spark-master:7077 --jars ... app.py`	Запуск на кластере Spark
`docker exec -it <spark-container> spark-submit --jars ... /app/job.py`	Запуск через Docker

11.3 PostgreSQL (psql)

Команда	Описание
`docker exec -it <pg> psql -U postgres -d mydb`	Подключиться к БД
`\dt`	Список таблиц
`\d table_name`	Описание таблицы
`SELECT count(*) FROM mock_data;`	Количество строк в staging

11.4 ClickHouse CLI

Команда	Описание
`docker exec -it <ch> clickhouse-client`	Подключиться к ClickHouse
`SHOW TABLES;`	Список таблиц
`DESCRIBE TABLE report_products;`	Описание таблицы
`SELECT * FROM report_products LIMIT 10;`	Просмотр данных
`SELECT count(*) FROM report_products;`	Количество строк
`curl 'http://localhost:8123/?query=SELECT+1'`	HTTP-запрос к ClickHouse

11.5 Полезные SQL (PostgreSQL, star schema)

```sql
-- Количество по всем таблицам
SELECT 'fact_sales' AS t, COUNT(*) FROM fact_sales UNION ALL
```

```sql
SELECT 'dim_customer', COUNT(*) FROM dim_customer UNION ALL
SELECT 'dim_seller', COUNT(*) FROM dim_seller UNION ALL
SELECT 'dim_product', COUNT(*) FROM dim_product UNION ALL
SELECT 'dim_store', COUNT(*) FROM dim_store UNION ALL
SELECT 'dim_supplier', COUNT(*) FROM dim_supplier;

-- Топ стран по выручке
SELECT c.customer_country, ROUND(SUM(f.sale_total_price), 2) AS revenue
FROM fact_sales f JOIN dim_customer c ON f.customer_id = c.customer_id
GROUP BY c.customer_country ORDER BY revenue DESC LIMIT 10;

-- Продажи по категориям
SELECT p.product_category, COUNT(*) AS cnt, ROUND(AVG(f.sale_total_price), 2) AS avg_price
FROM fact_sales f JOIN dim_product p ON f.product_id = p.product_id
GROUP BY p.product_category ORDER BY cnt DESC;
```

11.6 Полезные SQL (ClickHouse, витрины)

```sql
-- Обзор всех витрин
SELECT 'report_products', count() FROM report_products
UNION ALL SELECT 'report_customers', count() FROM report_customers
UNION ALL SELECT 'report_time', count() FROM report_time
UNION ALL SELECT 'report_stores', count() FROM report_stores
UNION ALL SELECT 'report_suppliers', count() FROM report_suppliers
UNION ALL SELECT 'report_quality', count() FROM report_quality;

-- Топ-10 продуктов
SELECT * FROM report_products ORDER BY total_sold DESC LIMIT 10;
```